

Users, Groups and Permissions

Linux File Security Overview

- Linux file security is the most basic access (authentication) and rights (authorization) management mechanism
- Standard Linux/UNIX security includes:
 - User and Password authentication
 - File & Directory access controland has several more advanced features

Linux/UNIX Accounts

- Each user has a unique ID (UID)
- Each user is a part of at least one group.
- Each group has a unique group ID (GID)
- There are three types of users:
 - **Super User:** also known as “root”, has full access to all the resources in the system without any restrictions; its UID is 0.
 - **Regular Users:** Normally have access to their own home-directory only; their UID’s will always be greater than 100.
 - **Pseudo Users:** Accounts that arrived built-into the system and do not reflect “real” users.

Users & Groups in Linux

- The system supports multiple users that have distinct properties and permissions.
- Linux defines groups to which a user can belong; groups add another level of file access permissions.
- A user can belong up to 16 different groups but can only belong to one primary group at any given time.
- The primary group of a user is applied as the “owning” group on any files or directories that user creates.

/etc/passwd File

- The **/etc/passwd** is a semicolon delimited file which lists and defines the system's user accounts.
- Each entry in the file represents a user account:
 - **nir:x:500:500:Nir:/home/nir:/bin/bash**
- Let's break down a user entry, from left to right:
 - 1) This is the username.
 - 2) This field is a representation of the legacy password field; in modern systems, the passwords are kept encrypted in **/etc/shadow** instead of as plain text in **/etc/passwd**.
 - 3) The account's UID.
 - 4) The account's primary GID.
 - 5) The account's comment section.
 - 6) The account's home-directory location.
 - 7) A command to execute upon user log-in; normally, this section is used to set the account's default shell, as seen in this example.

/etc/shadow File

- The **/etc/shadow** file holds the account passwords and their related settings:
 - **test:\$1\$0ifwRlGr\$SrDXfaxnvcoFUmR0IPW7a0:15172:0:99999:7:::**
- The entry broken down, left to right:
 - 1) This is the username.
 - 2) The encrypted password.
 - 3) Last password change; the measure here is in days since January 1st, 1970 which is the first day of the UNIX-time count.
 - 4) The minimum number of days required to pass before a user can change their password again.
 - 5) The maximum number of days a password is valid for and before the system forces the user to change it.
 - 6) The number of days before the password expires in which the system issues a warning to the user about the upcoming expiry.
 - 7) The number of days after password expiry after which the account becomes disabled.
 - 8) Days since June 1st, 1970 after which the account may no longer be used.

/etc/group File

- The **/etc/group** file contains the groups of the system, defines their GID's and member user accounts for each group.
 - **test:x:503:**
- Entry explained:
 - 1) The group's name.
 - 2) Password, generally unused unless a privileged group is required.
 - 3) GID.
 - 4) Member usernames, separated by a comma (,)
- There are two ways a user can be assigned to group(s):
 - The group number that appears in the 4th section of the **/etc/passwd** file entries; this group is also known as the Primary group for the account.
 - Type the user name(s) in the 4th section of the entry; the group will then become an additional group that user is member of, in addition to the fundamentally required primary group, listed in **/etc/passwd**.

User & Group Manipulation

- There are a few tools that allow us to manipulate users in manners of creation, editing and/or removal:
 - **useradd**: This command is used to create new users.
 - **usermod**: This one is used to modify existing users.
 - **userdel**: Deletes existing users.
- “**useradd**” has the ability to set every single property found in the **/etc/passwd** file entries upon creation of a new user; if no properties are explicitly specified, it will use the defaults which can be viewed by running: “**useradd -D**”.
- Very much like users, there are tools for group manipulation:
 - **groupadd**
 - **groupmod**
 - **groupdel**

Initializaing Users

- When a new user is created, all the files from within **/etc/skel** are copied into the new user's home-directory.
- The sys-admin can edit, customize and create files like **.bash_profile** and/or **.bashrc**, amongst others, that once a new user is created – they would automatically have a pre-defined, working environment which is not necessarily the default basic one.
- Note, once a user has been created and the files were copied from **/etc/skel** to his home directory, the only way to change them would be to edit them directly in that specific user's home directory.
- Important environment variables such as PATH should be set system-wide using **/etc/profile**

Changing User Passwords

- Aside from the users file, **/etc/passwd**, there is also a command named “**passwd**”.
- “**passwd**” is used to change user passwords.
- In order to change the password of the currently logged-on user, just type **passwd** and hit enter.
- We’ll be prompted for the current password then the new password we wish to have and a new password re-type verification.
- While logged on as the “root”, we are able to change password for any user we wish by running: “**passwd [username]**”.

File Ownership

- Each file and/or directory in Linux is owned by a single user and belongs to a single group.
- The ownership details are assigned at the time the file or directory are created.
- Note that user and group ownerships are distinct; it is possible for a user to own a file but not be a member of the owning group.
 - `-rwxrwxr-- 1 user1 group1 35 Jul 19 13:42 file2`
- The user ownership is colored in green and the group ownership in light-blue in the above example.

Access Modes

- There are three access modes:
 - Read, designated “r”
 - Write, designated “w”
 - Execute, designated “x”
- The meanings of the above access modes differ for files and directories:
 - **Files:**
 - **Read:** Access to view the file’s contents.
 - **Write:** Access to change the contents.
 - **Execute:** Access to execute the file (binary or shell script).
 - **Directories:**
 - **Read:** Access to view the directory’s contents.
 - **Write:** Access to change the directory’s contents (create or delete files)
 - **Execute:** Access to enter the directory (with the “cd” command).

Access Modes

- Every file and directory are affected by 3 sets of the above access modes:
 - `-rwxrwxrwx 1 nir test 35 Jul 19 13:42 file2`
- The first set (green) refers to user access, in this example's case the owning user is "nir".
- The second set (red) refers to group access, "test" in this case; all members of the group "test" are currently allowed to read, write and execute the file.
- The third set (blue) refers to "other" which affects any user or group that are not explicitly set as one of the owners.
- In the above example, anyone and everyone can read, write and execute the file.

Changing Ownerships

- By default, only the super-user (root) can change ownerships for files and/or directories.
- In order to change Group ownership only, we'd use the following command:
 - **chgrp [groupname] [filename(s)]**
- If we wish to change both user and group ownerships, we'd use:
 - **chown [username]:[groupname] [filename(s)]**

Changing Access Modes

- The only ones allowed to change access modes on files and directories are the owners and the super-user (root).
- The “**chmod**” command is used to change access modes; there are two methods of usage:
 - **Symbolic Mode**: uses a combination of letters and symbols to add or remove access permissions.
 - **Octal Mode**: Also known as Absolute or Numeric mode; this mode uses octal numbers that represent the different permissions in order to add or remove them.

Symbolic “chmod”

- The command’s syntax is:
 - **chmod [who][operation][permission(s)] [filename(s)]**
- List of “who”:
 - **a**: all; this includes user, group and other.
 - **u**: user.
 - **g**: group.
 - **o**: other.
- List of operations:
 - **+** : add permission, for example: **chmod u+r /tmp/test/file**
 - **-** : remove permission, **chmod g-x /tmp/test/file**
 - **=** : match permissions, **chmod a=rw /tmp/test/***

Octal (Absolute) “chmod”

- The command’s syntax is:
 - **chmod [octal mode] [filename(s)]**
- The octal modes are:
 - **Read: 4**
 - **Write: 2**
 - **Execute: 1**
- Any combination of the above numbers would set the file’s permissions:
 - **644** = rw-r--r--
 - **755** = rwxr-xr-x
 - **700** = rwx-----
 - **777** = rwxrwxrwx

Setting access modes with umask

- The “umask” filter determines the default permissions for newly created files and folders.
- Display the currently set umask by running: “**umask**”:
 - **# umask**
0002
- The digits in the umask value represent permissions that are to be “masked-out” from the maximum values of “777”; the masked permissions will **Not** be used when a new file or directory are created.
- This setting can be changed temporarily for the current session by running: “**umask [octal value]**”
- In order to make the umask change permanent, it must be added into the user’s initialization files.

Advanced Permissions - SUID

- SUID or SetUID is an additional permission bit that can be added to files or directories.
- When running an application or a shell script in Linux, the program will have the same permissions and access rights to the system as the user who executed it does.
- Some applications require elevated permissions so that they can access system files to achieve the desired results, however we as administrators, do not want to grant special permissions to regular users.
- This is when SUID comes in handy; it can be assigned to the executable program or script and when those run, by any user, the program would have elevated permissions, similar to a super-user's permissions.

Advanced Permissions - SUID

- Very important note: SUID is to be given ONLY to programs you know exactly what they are and trust them completely.
- Keep in mind that super-user permissions give complete control over the entire system and its contents to the user and/or application holding them.
- To apply SUID on a file or directory, run “**chmod**” with an additional number at the beginning of the octal permissions value:
 - `# chmod 4422 file_list`
`# ls -l | grep file_list`
`-r-S-w--w- 1 nir test 336 Jul 20 10:47 file_list`
- The upper case “**S**” is the SUID flag.
- To remove SUID, run the same chmod command with 0 instead of 4 as the first number in the octal value.

Introduction to Linux ACL

- “ACL” stands for “Access Control List”.
- ACL can be applied on files and directories in the system and are an addition to the standard User/Group/Other “rwx” permission model.
- ACL give another level of control over who can read, write and execute files.
- Linux kernel v2.6 and higher supports ACL for numerous file-system types:
 - EXT3
 - EXT2
 - XFS
 - JFS
 - ReiserFS

Introduction to Linux ACL's

- A pre-requisite for using ACL is that the files-ystem we wish to apply ACLs on is mounted with the “***acl***” option enabled.
- The commands used when setting and displaying ACL information are:
 - **getfacl**: display ACL settings
getfacl filename
 - **setfacl**: set acl settings
setfacl [options] [filename(s)]

Introduction to Linux ACL's

- *setfacl* options
 - **-m *type:name:rwx*** add permission of '***rwx***' for user or group '***name***'. '***t***' should be '***u***' for user, '***g***' for group or '***m***' in order to set the mask for this file
 - **-M *file*** adds permission according to the information in '***file***' (this file should in '***getfacl***' format)
 - **-x *type:name*** remove permissions to user or group '***name***'
 - **-b** removes all of the permission records on ACL

Introduction to Linux ACL's

```
# getfacl file1
file: file1
owner: root
group: root
user::rw-
group::r--
other::r--
```

```
# setfacl -m u:user1:rwx file1
# getfacl file1
file: file1
owner: root
group: root
user::rw-
user:user1:rwx
group::r--
mask::rwx
other::r--
```

```
# setfacl -m m::r-- file1
# getfacl file1
# file: file1
# owner: root
# group: root
user::rw-
user:user1:rwx      #effective:r--
group::r--
mask::r--
other::r--
```